

Design by Contract

O artigo de Bertrand Meyer apresenta uma abordagem para a confiabilidade de software baseada em uma metáfora: a construção de software como uma sucessão de decisões contratuais documentadas. Nessa perspectiva, cada rotina (função ou procedimento) estabelece um contrato com seu chamador (cliente), definindo obrigações e benefícios para ambas as partes. O autor argumenta que, para construir software correto e robusto, é necessário tornar explícitas as condições que devem ser satisfeitas antes e depois da execução de uma rotina – respectivamente, pré-condições e pós-condições. Porém, Meyer alerta que a simples existência de mecanismos de exceção não é suficiente; é preciso uma metodologia disciplinada, como a proposta pela linguagem Eiffel, que integra contratos ao próprio tecido da linguagem de programação.

A ideia central é simples e poderosa. Imagine que você está desenvolvendo um sistema de gerenciamento de tabelas (como uma tabela hash). A rotina de inserção put pode exigir que a tabela não esteja cheia antes de inserir um novo elemento – esta é a pré-condição. Em troca, após a execução, ela garante que o elemento foi inserido e que a contagem de elementos aumentou – esta é a pós-condição. Se o cliente chamar put com a tabela cheia, violou o contrato; a rotina não tem obrigação de funcionar corretamente nesse caso. Meyer compara essa relação a um contrato de construção civil: o cliente fornece o terreno (pré-condição) e o construtor entrega o edifício (pós-condição). Esse enfoque, conhecido como "programação defensiva reversa", transfere a responsabilidade de verificação para quem realmente tem o contexto para tal, evitando redundâncias que tornam o software complexo e frágil.

Um dos grandes benefícios dessa abordagem é a clareza na atribuição de responsabilidades durante o teste e a depuração. Se uma pré-condição é violada em tempo de execução, o erro é do cliente (quem chamou a rotina). Se uma pós-condição é violada, o erro é do fornecedor (a rotina que foi chamada). Isso simplifica imensamente a localização de bugs. Meyer exemplifica com a pilha (stack): a operação pop (remover elemento) deve ter uma pré-condição de que a pilha não está vazia. Se um cliente chamar pop sem verificar antes, a falha é dele. Isso contrasta com abordagens tradicionais que usam exceções para tratar pilha vazia, mas sem um contrato explícito, a responsabilidade fica vaga. O autor ainda destaca que a documentação gerada automaticamente a partir das asserções (pré e pós-condições) fornece uma especificação precisa e concisa do comportamento esperado, sem depender de implementação.

Apesar das vantagens, o artigo aponta desafios significativos, especialmente na implementação de mecanismos de exceção. Meyer critica duramente a abordagem do Ada (linguagem de programação) por permitir que uma rotina capture uma exceção, realize alguma ação e retorne ao chamador como se nada tivesse acontecido, sem sinalizar que o contrato foi quebrado. O exemplo dado é uma rotina de raiz quadrada que, ao receber um número negativo, imprime uma mensagem e

retorna um valor qualquer – o chamador não tem como saber que o cálculo não foi realizado. Essa prática é considerada perigosa, pois pode levar a resultados incorretos sem que ninguém perceba. Meyer propõe uma abordagem mais disciplinada, implementada na linguagem Eiffel, com duas cláusulas: *rescue* (resgate), que permite tentar uma estratégia alternativa (como pedir novo dado ao usuário) ou falhar de forma organizada, e *retry* (tentar novamente), que reinicia a rotina. O autor também apresenta um exemplo de leitura de inteiro com até cinco tentativas, mostrando como a solução em Eiffel é mais clara e segura que a equivalente em Ada.

Outro ponto importante abordado é a herança e a redefinição de rotinas em linguagens orientadas a objetos. Meyer introduz o conceito de "subcontratação honesta": uma classe filha que redefine uma rotina da classe pai pode enfraquecer a pré-condição (exigir menos do cliente) e fortalecer a pós-condição (garantir mais resultados), mas nunca o contrário. Isso é formalizado com as cláusulas *require else* (pré-condição alternativa) e *ensure then* (pós-condição adicional). O autor exemplifica com uma tabela que, na versão original, tinha pré-condição de não estar cheia. Uma versão melhorada que redimensiona automaticamente pode ter pré-condição mais fraca (*true*, ou seja, sempre aceita). Isso respeita o contrato original: quem chama a versão melhorada recebe o mesmo ou mais do que esperava. Essa regra evita que herança seja usada de forma arbitrária, garantindo que o comportamento polimórfico (onde uma variável da classe pai pode referenciar um objeto da classe filha) seja previsível e seguro.

Em suma, o artigo de Bertrand Meyer oferece uma base sólida para a construção de software confiável por meio da metáfora do contrato. Ficou evidente que a definição explícita de pré-condições, pós-condições e invariantes de classe, aliada a um mecanismo disciplinado de exceções (com *rescue* e *retry*), permite atribuir responsabilidades de forma clara durante o desenvolvimento, teste e manutenção. O tratamento de exceções em Ada é criticado por permitir que rotinas falhem silenciosamente, enquanto a abordagem do Eiffel força a sinalização de falhas ou a tentativa de recuperação de forma controlada. Além disso, as regras para redefinição em herança (*require else* e *ensure then*) garantem que o polimorfismo não introduza comportamentos inesperados. Diante disso, o projeto por contrato mostra-se não apenas uma técnica teórica, mas uma ferramenta prática e poderosa para aumentar a robustez e a clareza de sistemas de software, especialmente quando integrada à própria linguagem de programação.

Referência Bibliográfica

MEYER, Bertrand. **Design by Contract**. In: ... (capítulo de livro). Disponível em: <https://github.com/joaopauloaramuni/projeto-de->

[software/blob/main/ARTIGOS/Design-by-Contract.pdf](#). Acesso em: 13 de Maio de 2026.